Tugas Studi Kasus : Perpustakaan Kampus
Rekayasa Perangkat Lunak

Anggota Kelompok:
Aditya Widyatamaka Tri Utomo (250535623587)
Chasbiah Azzahra (250535621731)
Dzauq Bachrul ‘Ulum (250535621835)
Elshifa Viorissa (250535621859)

Soal:

Skenario	Tentukan
Mahasiswa login	Actor
Mencari buku	Use Case
Melihat ketersediaan	Satu Activity DIagram
Melakukan peminjaman	Class utama
Menerima informasi status peminjaman	Hubungan antar-model
Dapat mengembalikan buku	requirement yang menjadi dasar setiap model

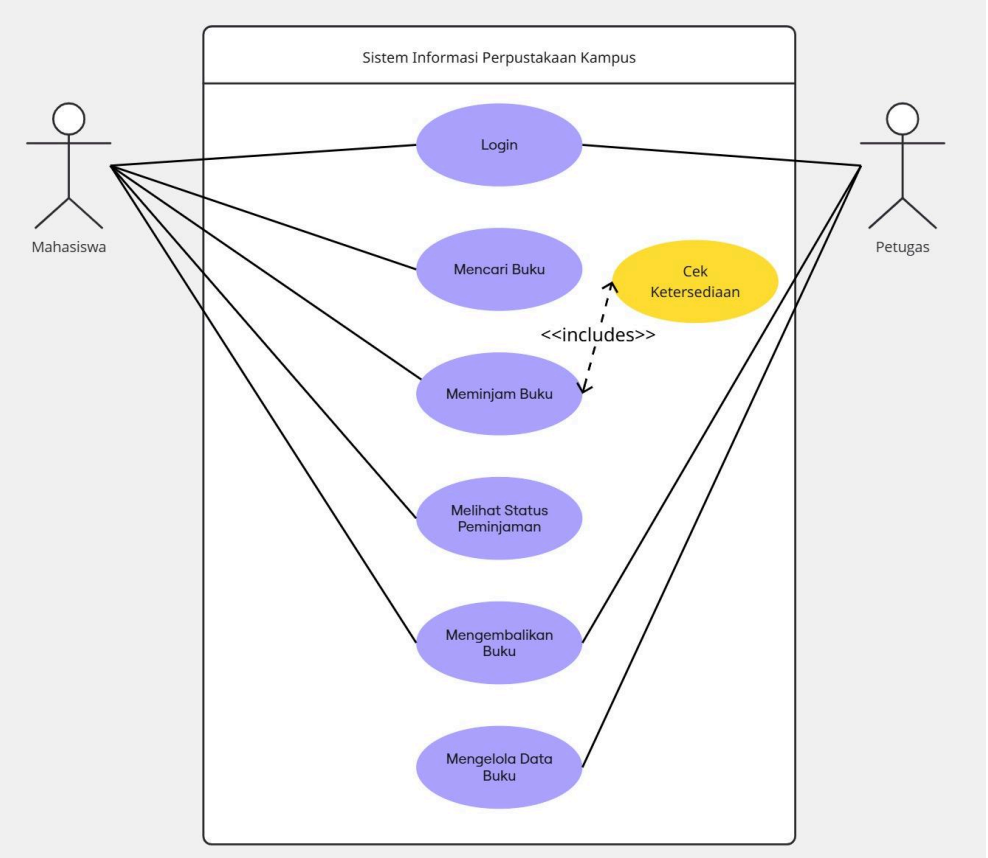
Pertanyaan pengembangan:
Bagaimana data dari software metrics dapat membantu mengevaluasi apakah proses pengembangan sistem ini berjalan dengan baik?

jawaban:

1. Actor
    - Mahasiswa (Actor Utama): Pengguna yang berinteraksi langsung dengan sistem untuk melakukan login, mencari buku, mengecek ketersediaan, melakukan peminjaman, melihat status peminjaman, serta mengembalikan buku.
    - Petugas Perpustakaan (Actor Pendukung): Mengelola data buku dan memvalidasi/memproses transaksi pengembalian.

2. Use Case
    - Login: Mengakses akun mahasiswa ke dalam sistem.
    - Mencari Buku: Melakukan pencarian katalog buku.
    - Cek Ketersediaan: Memeriksa stok buku yang dicari (<<include>> pada proses peminjaman).
    - Meminjam Buku: Mengajukan dan memproses transaksi peminjaman buku.

- Melihat Status Peminjaman: Menerima dan memantau status peminjaman buku.
- Mengembalikan Buku: Memproses pengembalian buku yang telah dipinjam.



3. Activity diagram alur peminjaman dan pengembalian
   Alur Peminjaman:
   Mahasiswa *login* → Mencari buku pada katalog sistem → Sistem menampilkan daftar dan ketersediaan buku → Mahasiswa memilih buku untuk dipinjam → Sistem memvalidasi batas kuota pinjam mahasiswa → Jika valid, sistem mencatat transaksi peminjaman, mengubah status eksemplar buku menjadi "Dipinjam", dan menampilkan informasi status peminjaman serta tenggat pengembalian.
   Alur Pengembalian:
   Mahasiswa menyerahkan buku kepada petugas → Petugas memindai kode/ID buku pada sistem → Sistem menghitung selisih hari dari tenggat waktu untuk memeriksa keterlambatan → Jika terlambat, sistem mengkalkulasi denda → Sistem memperbarui status eksemplar buku kembali menjadi "Tersedia" dan menutup transaksi peminjaman.

4. Class utama
   - Mahasiswa:
     - *Atribut:* nim, nama, jurusan, statusAkunAktif
     - *Method:* login(), ajukanPinjam()
   - Buku (Buku Induk):

    - *Atribut:* idBuku, judul, penulis, kategori, tahunTerbit
    - *Method:* getDetail()
- EksemplarBuku (Unit Fisik Buku):
    - *Atribut:* idEksemplar, kondisiBuku, statusKetersediaan (Tersedia / Dipinjam)
    - *Method:* ubahStatus()
- Peminjaman:
    - *Atribut:* idPeminjaman, tanggalPinjam, tanggalJatuhTempo, tanggalKembali, statusTransaksi
    - *Method:* hitungDenda(), tutupTransaksi()
- Petugas:
    - *Atribut:* idPetugas, nama
    - *Method:* validasiPeminjaman(), catatPengembalian()

5. Ketiga model dalam *Analysis Modeling* (Use Case Diagram, Activity Diagram, dan Class Diagram) saling melengkapi untuk memberikan gambaran sistem yang utuh:
    - Use Case Diagram (Fungsional) menentukan APA yang dapat dilakukan oleh sistem dan siapa aktor yang berinteraksi.
        - *Contoh:* Use Case "Meminjam Buku" dan "Melihat Ketersediaan".
    - Activity Diagram (Perilaku/Behavioral) menjelaskan BAGAI MANA alur kerja (*workflow*) dan interaksi langkah demi langkah dari satu atau beberapa Use Case.
        - *Contoh:* Saat mahasiswa melakukan "Meminjam Buku", Activity Diagram menggambarkan alur memilih buku → pengecekan stok → konfirmasi → menyimpan transaksi peminjaman.
    - Class Diagram (Struktural) menyediakan ENTITAS/DATA yang dibutuhkan agar proses dalam Activity Diagram dan Use Case dapat berjalan.
        - *Contoh:* Proses pembuatan transaksi pada Activity Diagram memerlukan class Mahasiswa, class Buku (dengan atribut stok), dan class Peminjaman.

    Ringkasan Keterkaitan: Use Case mendefinisikan fitur, Activity Diagram mendeskripsikan jalannya fitur tersebut, dan Class Diagram menyediakan struktur data/objek untuk menopang jalannya fitur.

6. Requirement yang Menjadi Dasar Setiap Model
    Pemodelan harus memiliki *traceability* (keterusuran) yang jelas terhadap skenario/requirement pendukungnya:

    - Use Case Diagram
        - *Login* ← Skenario 1
        - *Mencari Buku* ← Skenario 2
        - *Cek Ketersediaan* ← Skenario 3

    - *Meminjam Buku* ← Skenario 4 & 5 (Melakukan peminjaman dan menerima informasi status peminjaman)
    - *Mengembalikan Buku* ← Skenario 6
- Activity Diagram
    - Berdasarkan gabungan alur Skenario 1 s.d. 5: Diawali dari mahasiswa login → cari buku → lihat ketersediaan → jika stok ada, proses peminjaman disimpan → tampilkan/terima informasi status peminjaman.
- Class Diagram
    - Class Mahasiswa ← Dasar dari Skenario 1 & 4 (Aktor yang login dan melakukan peminjaman).
    - Class Buku ← Dasar dari Skenario 2 & 3 (Data buku yang dicari dan dicek ketersediaan/stoknya).
    - Class Peminjaman ← Dasar dari Skenario 4, 5, & 6 (Pencatatan transaksi peminjaman, status peminjaman, dan pengembalian buku).

7. Dampak Perubahan Requirement Terhadap Model (Change Impact Analysis) Jika terjadi perubahan kebutuhan (Requirement Change), misalnya:

   Contoh Kasus Perubahan:
   "Kampus menambahkan fitur reservasi/booking buku secara online maksimal 1x24 jam sebelum diambil di perpustakaan, serta sistem sanksi otomatis berupa penangguhan akun jika terlambat mengembalikan lebih dari 7 hari."

   Analisis Dampak Perubahan:
   1. Dampak pada Use Case Diagram:
      - Muncul Use Case baru: UC-08: Reservasi Buku yang terhubung dengan Aktor Mahasiswa.
      - Muncul relasi <<extend>> dari Cek Ketersediaan Buku ke Reservasi Buku (jika buku habis, mahasiswa dapat masuk ke daftar antrean/booking).
   2. Dampak pada Activity Diagram:
      - Pada alur peminjaman, ditambahkan alur validasi baru untuk memeriksa apakah buku yang diambil berstatus "Sedang Direservasi oleh mahasiswa lain".
      - Ditambahkan timer/trigger 24 jam untuk pembatalan reservasi secara otomatis.
      - Pada alur pengembalian, ditambahkan decision branch: jika keterlambatan > 7 hari, sistem mengeksekusi aksi penangguhan akun.
   3. Dampak pada Class Diagram
      - Penambahan Class baru: Reservasi (idReservasi, waktuBooking, batasAmbil, statusBooking).
      - Penambahan Atribut & Method:

        - Di Mahasiswa: atribut statusAkunAktif (boolean), method tangguhkanAkun().
        - Di EksemplarBuku: penambahan status baru "Direservasi".
    - Penambahan Relasi: Relasi Mahasiswa (1) -- (0..*) Reservasi dan Buku (1) -- (0..*) Reservasi.
4. Pentingnya Traceability Matrix
    - Dengan matriks ketertelusuran, tim pengembang dapat melakukan Ripple Effect Analysis (analisis efek domino), sehingga modul yang terpengaruh dapat diisolasi dan diuji ulang (regression testing) tanpa merusak fungsi yang sudah stabil.

8. Pertanyaan Pengembangan: Pemanfaatan Software Metrics
   Pertanyaan: Bagaimana data dari software metrics dapat membantu mengevaluasi apakah proses pengembangan sistem ini berjalan dengan baik?

   Data dari Software Metrics (Metrik Perangkat Lunak) berfungsi sebagai instrumen kuantitatif berbasis data (data-driven) untuk memonitor, mengontrol, dan mengevaluasi kesehatan proyek perangkat lunak dari tiga dimensi utama:

   1. Evaluasi Proses (Process Metrics)
        - Defect Arrival Rate & Defect Density
            - Fungsi: Mengukur jumlah bug/cacat yang ditemukan pada setiap tahapan (analisis, desain, coding, dan pengujian).
            - Indikator: Jika jumlah bug menurun menjelang fase deployment, proses pengembangan berjalan dengan matang. Jika bug masih tinggi pada fase akhir, berarti tahapan verifikasi modul awal masih lemah.
        - Sprint Velocity / Burn-down Rate (jika memakai Agile/Scrum):
            - Fungsi: Mengukur kesesuaian estimasi story points atau effort tim dengan realisasi fitur yang berhasil diselesaikan (sesuai dengan Definition of Done).
            - Indikator: Grafik burn-down yang stabil menunjukkan kapasitas tim yang dapat diprediksi dengan baik.

   2. Evaluasi Kualitas Produk & Kode (Product & Code Metrics)
        - Cyclomatic Complexity (Metrik McCabe):
            - Fungsi: Mengukur tingkat kompleksitas logika percabangan (decision paths) pada method sistem (misalnya logika validasi denda dan stok).
            - Indikator: Nilai Cyclomatic Complexity yang ideal adalah ≤ 10. Jika lebih dari itu, fungsi tersebut rentan terhadap bug dan sulit dibuatkan unit test, sehingga perlu direfaktor.
        - Object-Oriented Design Metrics (Chidamber & Kemerer):
            - CBO (Coupling Between Objects): Mengukur seberapa saling tergantung antar kelas. Nilai CBO yang rendah (loose coupling) menandakan arsitektur sistem yang modular dan mudah dirawat.

        - LCOM (Lack of Cohesion in Methods): Mengukur kohesivitas kelas. Nilai kohesi yang tinggi (high cohesion) menunjukkan bahwa setiap kelas memiliki tanggung jawab tunggal (Single Responsibility Principle).
    - Code Coverage (Test Coverage):
        - Fungsi: Mengukur persentase baris kode yang dieksekusi oleh automated test suite.
        - Indikator: Coverage di atas 80% menunjukkan pengujian yang menyeluruh dan meminimalkan risiko regresi saat menambahkan fitur baru.

3. Evaluasi Kestabilan Spesifikasi (Project & Requirement Metrics)
    - Requirement Volatility Index (RVI):
        - Fungsi: Mengukur persentase perubahan, penambahan, atau pembatalan persyaratan selama siklus pengembangan.
        - Indikator: RVI yang terlalu tinggi mengindikasikan bahwa fase komunikasi dan elicitation awal belum matang, sehingga berisiko menyebabkan keterlambatan jadwal (schedule slippage) dan pembengkakan biaya (budget overrun).

Kesimpulan: Melalui data metrik perangkat lunak, manajer proyek dan tim developer tidak mengandalkan perkiraan subjektif (feeling), melainkan memiliki visibilitas objektif untuk mendeteksi bottleneck lebih awal, memastikan kode mudah dipelihara (maintainable), serta menjamin sistem perpustakaan kampus selesai tepat waktu dengan kualitas yang teruji.